CO3- LAB EXPERIMENTS

1. Prolog Program – Towers of Hanoi

Code

```
hanoi(1, From, To, _) :-
   write('Move disk from '),
   write(From),
   write(' to '),
   write(To),
   nl.


hanoi(N, From, To, Aux) :-
   N > 1,
   N1 is N - 1,
   hanoi(N1, From, Aux, To),
   hanoi(1, From, To, Aux),
   hanoi(N1, Aux, To, From).
```

Input

```
?- hanoi(3, left, right, center).
```

Output

```
Move disk from left to right
Move disk from left to center
Move disk from right to center
Move disk from left to right
Move disk from center to left
Move disk from center to right
Move disk from left to right


true.
```

---

2. Prolog Program – Bird Can Fly or Cannot Fly

Code

```
bird(parrot).
bird(eagle).
bird(penguin).
bird(ostrich).

can_fly(parrot).
can_fly(eagle).

cannot_fly(X) :-
   bird(X),
   \+ can_fly(X).

check_bird(X) :-
   can_fly(X),
   write(X),
   write(' can fly.'), nl.

check_bird(X) :-
   cannot_fly(X),
   write(X),
   write(' cannot fly.'), nl.
```

Input

```
?- check_bird(parrot).
?- check_bird(penguin).
?- check_bird(eagle).
```

?- check_bird(ostrich).

Output

parrot can fly.

penguin cannot fly.

eagle can fly.

ostrich cannot fly.

---

3. Prolog Program – Family Tree

Code

```
parent(john, alice).
parent(john, bob).
parent(mary, alice).
parent(mary, bob).

parent(bob, charlie).
parent(susan, charlie).

father(X, Y) :-
   parent(X, Y),
   X = john.

mother(X, Y) :-
   parent(X, Y),
   X = mary.
```

```
sibling(X, Y) :-
    parent(P, X),
    parent(P, Y),
    X \= Y.

grandparent(X, Z) :-
    parent(X, Y),
    parent(Y, Z).
```

Input

```
?- parent(john, alice).
?- sibling(alice, bob).
?- grandparent(john, charlie).
```

Output

```
true.

true.

true.
```

---

4. Prolog Program – Dieting System Based on Disease

Code

```
diet(diabetes, 'Eat vegetables, whole grains and avoid sugar').
diet(hypertension, 'Eat fruits, vegetables and reduce salt').
diet(obesity, 'Eat low calorie food and exercise regularly').
diet(anemia, 'Eat spinach, beans and iron rich food').

suggest_diet(Disease) :-
    diet(Disease, Diet),
```

```
    write('Recommended Diet: '),
    write(Diet),
    nl.
```

Input

```
?- suggest_diet(diabetes).
?- suggest_diet(hypertension).
?- suggest_diet(obesity).
```

Output

Recommended Diet: Eat vegetables, whole grains and avoid sugar

Recommended Diet: Eat fruits, vegetables and reduce salt

Recommended Diet: Eat low calorie food and exercise regularly

---

5. Prolog Program – Monkey Banana Problem

Code

```
move(state(middle, on_floor, middle, has_not),
    grasp,
    state(middle, on_floor, middle, has)).

move(state(P, on_floor, P, Has),
    climb,
    state(P, on_box, P, Has)).

move(state(P, on_floor, B, Has),
    walk(P, B),
    state(B, on_floor, B, Has)).
```

```
move(state(P, on_floor, B, Has),
     push(P, B),
     state(B, on_floor, B, Has)).


solve(state(_, _, _, has)).


solve(State) :-
   move(State, Action, NewState),
   write(Action),
   nl,
   solve(NewState).
```

Input

```
?- solve(state(left, on_floor, middle, has_not)).
```

Output

```
walk(left,middle)
climb
grasp


true.
```

---

6. Prolog Program – Fruit and Color using Backtracking

Code

```
fruit_color(apple, red).
fruit_color(banana, yellow).
fruit_color(orange, orange).
fruit_color(grapes, green).
fruit_color(mango, yellow).
```

```
show_fruits :-
   fruit_color(Fruit, Color),
   write(Fruit),
   write(' is '),
   write(Color),
   nl,
   fail.

show_fruits.
```

Input

```
?- fruit_color(Fruit, Color).
```

Output

```
Fruit = apple,
Color = red ;

Fruit = banana,
Color = yellow ;

Fruit = orange,
Color = orange ;

Fruit = grapes,
Color = green ;

Fruit = mango,
Color = yellow.
```

---

7. Prolog Program – Best First Search

Code

```
edge(a, b, 2).
edge(a, c, 5).
edge(b, d, 4).
edge(b, e, 3).
edge(c, f, 2).
edge(e, g, 1).
edge(d, g, 5).
edge(f, g, 3).

heuristic(a, 7).
heuristic(b, 5).
heuristic(c, 4).
heuristic(d, 5).
heuristic(e, 2).
heuristic(f, 3).
heuristic(g, 0).

best_first(Start, Goal, Path) :-
   search([Start], Goal, Path).

search([Goal|_], Goal, [Goal]).

search([Current|Rest], Goal, [Current|Path]) :-
   findall(H-Next,
          (edge(Current, Next, _),
           heuristic(Next, H),
           \+ member(Next, Rest)),
```

```
        Neighbors),
    sort(Neighbors, Sorted),
    extract_nodes(Sorted, Nodes),
    append(Nodes, Rest, NewQueue),
    search(NewQueue, Goal, Path).

extract_nodes([], []).

extract_nodes([_-Node|T], [Node|R]) :-
    extract_nodes(T, R).
```

Input

```
?- best_first(a, g, Path).
```

Output

```
Path = [a, c, f, g].
```

---

8. Prolog Program – Medical Diagnosis

Code

```
symptom(ravi, fever).
symptom(ravi, cough).
symptom(ravi, headache).

symptom(anu, fever).
symptom(anu, body_pain).

symptom(kumar, cough).
symptom(kumar, cold).

diagnosis(Person, flu) :-
```

```
    symptom(Person, fever),
    symptom(Person, cough).

diagnosis(Person, common_cold) :-
    symptom(Person, cough),
    symptom(Person, cold).

diagnosis(Person, viral_fever) :-
    symptom(Person, fever),
    symptom(Person, body_pain).

check_diagnosis(Person) :-
    diagnosis(Person, Disease),
    write(Person),
    write(' may have '),
    write(Disease),
    nl.
```

Input

```
?- check_diagnosis(ravi).
?- check_diagnosis(anu).
?- check_diagnosis(kumar).
```

Output

```
ravi may have flu.

anu may have viral_fever.

kumar may have common_cold.
```

---

9. Prolog Program – Forward Chaining

Code

```
fact(sunny).

fact(warm).


rule(sunny, outdoor).

rule(warm, comfortable).

rule(outdoor, play).


forward :-

   fact(X),

   rule(X, Y),

   \+ fact(Y),

   assertz(fact(Y)),

   forward.


forward.
```

Input

```
?- forward.

?- fact(outdoor).

?- fact(play).
```

Output

```
true.


true.


true.
```

---

10. Prolog Program – Backward Chaining

Code

```
fact(bird).

fact(has_feathers).


rule(can_fly, bird).

rule(can_fly, has_feathers).


prove(X) :-

   fact(X).


prove(X) :-

   rule(X, Y),

   prove(Y).
```

Input

```
?- prove(bird).

?- prove(has_feathers).

?- prove(can_fly).
```

Output

```
true.


true.


true.
```